Foundational Competencies and Responsibilities of a Research Software Engineer - A summary

The teachingRSE project [1]

[1] https://github.com/the-teachingRSE-project

2024-10-10

Abstract: The term Research Software Engineer, or RSE, emerged a little over 10 years ago as a way to represent individuals working in the research community but focusing on software development. The term has been widely adopted and there are a number of high-level definitions of what an RSE is. However, the roles of RSEs vary depending on the institutional context they work in. At one end of the spectrum, RSE roles may look similar to a traditional research role. At the other extreme, they resemble that of a software engineer in industry. Most RSE roles inhabit the space between these two extremes. Therefore, providing a straightforward, comprehensive definition of what an RSE does and what experience, skills and competencies are required to become one is challenging. In this community paper we define the broad notion of what an RSE is, explore the different types of work they undertake, and define a list of foundational competencies as well as values that outline the general profile of an RSE. Further research and training can build upon this foundation of skills and focus on various aspects in greater detail. We expect that graduates and practitioners will have a larger and more diverse set of skills than outlined here. We look at specific types of RSE roles, propose recommendations for organisations, and give examples of future specialisations.

Keywords: research software engineering, RSE, competencies, curriculum design, teaching

---

Contents

1 Introduction **2**
1.1 Terminology . . . 3
1.1.1 The term Research Software Engineer . . . 3
1.1.2 Further definitions . . . 4

2 Values **4**

3 Foundational RSE competencies **5**
3.1 Software/Technical skills . . . 5
3.1.1 Classical software engineering skills . . . 6
3.1.2 Adapting to the software life cycle (SWLC) . . . 6
3.1.3 Creating documented code building blocks (DOCBB) . . . 6
3.1.4 Building distributable software (DIST) . . . 6
3.1.5 Use software repositories (SWREPOS) . . . 7
3.1.6 Software behaviour awareness and analysis (MOD) . . . 7
3.2 Research skills . . . 7
3.2.1 Conducting and leading research (NEW) . . . 7
3.2.2 Understanding the research cycle (RC) . . . 7
3.2.3 Software re-use (SRU) . . . 7
3.2.4 Software publication and citation (SP) . . . 7
3.2.5 Using domain repositories/directories (DOMREP) . . . 8
3.3 Communication skills . . . 8
3.3.1 Working in a team (TEAM) . . . 8
3.3.2 Teaching (TEACH) . . . 8
3.3.3 Project management (PM) . . . 8
3.3.4 Interaction with users and other stakeholders (USERS) . . . 8

4 RSE specialisations **9**
4.1 Specialisations within the core RSE competencies . . . 9
4.2 Specialisations outside the core RSE competencies . . . 9

5 Future work **11**

6 Conclusion **11**

Contribution details **12**

Acknowledgements **12**

References **12**

Glossary **14**

Skill codes **14**

Acronyms **14**

1 Introduction

Computers and software have played a key role in the research life cycle for many decades. They are now vital elements of the research process across almost all domains. They enable researchers to collect and process ever-increasing amounts of data, simulate a wide range of physical phenomena across previously unexplored scales of the universe, and discover previously inconceivably complex structures in nature and societies via machine learning (ML). This prevalence of computation and digitally-aided data analysis in research means that digital skills are now required by researchers at all career levels, and in fields significantly beyond those that would previously have been expected. Research software is now used and developed not only in science, technology, engineering and mathematics (STEM) domains, but also in other fields, like medicine and the humanities.

Researchers often lack the skills to use specialised software for their research, let alone write it [31]. If they come from a non-technical domain, they may also struggle to know what to ask when trying to request help from and interact with more experienced colleagues. A gap still exists in academic education, as many curricula do not sufficiently prepare students in this regard. This situation is exemplified by the extracurricular Massachusetts

Institute of Technology (MIT) class "The Missing Semester of Your CS Education" [29], which aims to increase "computing ecosystem literacy" even among students of Computer Science at MIT.

Researchers investing increasing amounts of their time developing their software engineering (SE) skills to support their research work can find themselves with little time to do the research itself. This, in turn, presents career development challenges since the experience required to gain and progress in research and academic roles is traditionally assessed through metrics that do not directly include software outputs. A recent shift towards the establishment of the distinct role of a *"Research Software Engineer"* [20] (RSE, a term that was coined in the United Kingdom (UK) a little over 10 years ago [15]), now provides a base on which sustainable career opportunities can be (and are being) built, allowing for better training of researchers and more effective support for the development of high quality research software. There is still a long way to go, but positive change is well underway.

RSEs may work within one of the increasing number of research software engineering teams that have been set up at universities and research organisations over the past decade, or they may be embedded within a research team. They may have a job title that officially recognises them as an RSE, or they may have a standard research or technical job title such as Research Assistant, Research Fellow, or Software Engineer. Regardless of their job title, RSEs share a set of core skills that are required to design and develop research software, understand the research environment, and ensure that they produce sustainable, maintainable code that supports reproducible research outputs, following the Findability, Accessibility, Interoperability and Reusability (FAIR) principles [1].

This community paper defines a set of core values and foundational competencies, which an RSE should acquire during training and formal education. These skills are formulated independently of a specific research domain and current technical tools used to support the application of the skills. By defining these competencies, we provide a guiding framework to facilitate the training and continuous professional development of RSEs, thus helping to provide a positive impact on research outputs and, ultimately, society as a whole. These competencies draw upon skills from traditional SE practice, established research culture, and the commitment to being part of a team. However, we see this set of skills as a foundation to build upon. We envision that through specialised training, the set of skills of graduate RSEs and domain researchers will grow. This is underlined by a growing interest to perform RSE research, i.e. research into methods and tools more catered to the unique challenges that research software provides.

While this community paper is based on workshop discussions that were attended largely by RSEs (deRSE23 in Paderborn, un-deRSE23 in Jena, and deRSE24 in Würzburg, all in Germany), we believe that the competencies formulated here can offer far-reaching impact beyond the domain of RSE into adjacent aspects of research and, indeed, the wider research community. This is especially important given that much research involves some amount of data management, processing and visualisation, or the creation of tools for these tasks, and funding bodies and computing infrastructure providers will sometimes prioritise projects that generate archived, annotated, re-usable, and potentially remotely executable data. In particular, funding agencies and research managers will find the discussion in this paper valuable in order to discover where RSEs see their place in the existing landscape of scientific domains and how to support the work of RSEs at different positions and career levels.

This paper is a condensed version of a more comprehensive paper published at FIXME:INSERTREFHERE . A non-exhaustive overview of similar initiatives can be found in FIXME:INSERTREFHERE . In this paper, we start to elaborate in Section 2 on the values that provide the guiding principles for the work of an RSE. Section 3 defines a set of core skills based on these values. We categorise these skills into three pillars, namely "software/technical", "research", and "communication" skills, reflecting the hybrid nature of an RSE. To justify the selection of these skills, we also list some current tasks and discuss the skills used therein.

As with any general skill set, not all RSEs will need to use all the skills highlighted to the same level of expertise. But the Discussion of how much a person needs to know depending on their education or career level or on the type of projects they would like to be involved with can be found in FIXME:INSERTREFHERE . Also, the overview of what skills and limitations an RSE in different team structures typically has, and recommendations for organisations that need to support RSEs is in expanded paper in FIXME:INSERTREFHERE . Section 4 provides a list of RSE specialisations and discusses the level of skill needed to work in each of them, before we conclude the paper with details of future work in Section 5 and conclusions in Section 6.

1.1 Terminology

1.1.1 The term Research Software Engineer

Research Software Engineering can be considered an interface discipline, linking traditional Software Engineering with Research itself [18]. Due to this nature there is a plethora of different variations of RSE depending on the

particular Research domain they are working in. Therefore the broad notion of Research Software Engineers is better thought of as a collection of sub-communities. The term Research Software Engineer is made more difficult to grasp since an internationally recognised definition is still missing. While there is consensus about the general notion that an RSE is a person with one leg in their research domain and the other in software development, this spans a whole spectrum depending on which one is more emphasised. There is also the question of what level of professionalism concerning both non-SE research and SE is expected. A more inclusive definition allows more people to self-identify as RSEs, thereby also fostering an inclusive community of people working in digital science (see also Section 2 on the values of an RSE). RSEs fall therefore somewhere on the spectrum between a researcher at one end and a software engineer at the other. Common to all of them is, that they need to be able to work in the research environment the software is used in, ideally at eye-level with native researchers, but at least as close as possible. RSEs often need to deal with non-technical complexities that are characteristic for research environments: organisational, motivational, with respect to the size of projects, independence and heterogeneous goals of stakeholders, boundary conditions for funding and future funding, to name just a few. Summarising, RSEs have skills and experience in three important areas: in the research area(s) their software is used in, in software engineering topics, as well as in interdisciplinary communication.

1.1.2 Further definitions

Depending on the national research environments and processes that readers are familiar with, the notion of the terms *software* and *research* might differ. Therefore, to avoid ambiguities, we define these as follows:

Software: Source code, documentation, tests, executables and all other artefacts that are created during the development process that are necessary to understand its purpose.

Research software: Foundational algorithms, the software itself, as well as scripts and computational workflows that were created during the research process or for a research purpose, across all domains of research. This definition is broader than in [1] and is the outcome of a recent discussion in [14].

Research software engineers: People who create or improve research software and/or the structures that the software interacts with in the computational environment of a research domain. They are highly skilled team members who may also choose to conduct their own research as part of their role. However, we also recognise that many RSEs have chosen specifically to focus on a technical role as an alternative to a traditional research role because they enjoy and wish to focus on the development of research software.

Researchers: People who are using the services provided by Research Software Engineers. This, on purpose, is a very broad definition and was chosen for easier readability.

2 Values

It is important that the activities of an RSE are guided by ethical values. In addition to the values for good scientific practice [9], RSEs also adhere to the SE Code of Ethics [13]. Central to that code is the RSE's obligation to In addition to the values for good scientific practice commit to the health, safety and welfare of the public and act in the interest of society, their employer and their clients. Further values loosely based on that code include the obligations

- to commit to objectivity and fact-based, honest research conclusions,
- to promote openness and accountability in the research process,
- to take great care to develop software that adheres to current best practices,
- to judge independently and maintain professional integrity,
- to treat colleagues and collaborators with respect and work towards a fair and inclusive environment, and
- to promote these values whenever possible and make sure that they are passed on to new practitioners.

The deployment of computer-based modelling and simulation has dramatically changed the practice of science in a large number of fields. It has enabled the hitherto impossible study of new classes of problems, often replacing traditional experimentation and observation. It can also serve to integrate a communal body of knowledge [22]. Humphreys [17] regards this development as "more important than the invention of the calculus in the 1660s, an event that remained unparalleled for almost 300 years". The epistemological status of computer modelling and simulation is still the subject of debate, which ranges from the postulate of a new process of knowledge creation that has its own, unique, epistemology [33] to the perception that from a philosophy of science perspective, there is nothing really new [10]. In any case, it is clear that a number of decisions in the construction of a simulation-model will have a significant impact on the adequacy for purpose [3] of the model. These decisions include the selection

of the salient characteristics of the system to be modelled, the choice of the mathematical representation of the processes to be represented, the choice of numerical methods and other algorithms and even including the design of the user-interface.

The relationship between initial state, inputs and final state of a computer simulation is "epistemically opaque" [17], in that not every step of the process is directly observable. The current trend of an increasing application of computationally irreducible systems, such as those based on artificial neural networks, further exacerbates this inherent limitation of explainability. An RSE usually takes a pivotal role in assessing this adequacy for purpose of a model as well as in characterising and communicating the domain of its legitimate application and its limits of interpretability. This role, together with the enormous reliance on modelling and simulation of scientific results, as well as real-world decision-making, places a large responsibility on the RSE. It is important that RSEs are aware of this responsibility and continuously improve their capabilities to live up to it.

Research software is also well on its way to being ever-present in data-driven research, in all research fields. This can probably be most prominently seen by considering software used to analyse data, e.g. within experimental research. It is not unusual for RSEs to support those more research data oriented efforts as well. Here, specifically, they closely interact with research data management professionals and practices by designing research software that is better able to adhere to the FAIR principles for research data, but also to follow similar rules for research software (FAIR4RS [1]). As such, they are then familiar with special requirements stemming from the field itself, e.g., in medical research, and with privacy related issues especially for personal data, e.g., for conducting surveys.

RSEs often assume a multifaceted role at the junction of research, SE and data management. They work with a varying and diverse set of colleagues that might include other developers, support unit staff and academics of different fields and all career stages. This situation yields a specific set of challenges RSEs should be aware of to consciously make ethically sound judgement calls. We list some example areas that highlight present-day challenges.

3 Foundational RSE competencies

The role of an RSE lies somewhere on the spectrum between that of a researcher (the "R") and a software engineer (the "SE") and, therefore, requires competencies in both fields. RSEs typically have a background in research or software engineering, but they definitely have obtained broader knowledge in both fields. Even when working as the only RSE on a task or project, they typically apply their knowledge and experience as part of larger teams of researchers and technical professionals, which allows them to cultivate this hybrid nature. There are many ways to categorise the competencies of an RSE. We chose to distribute these competencies over three pillars to reflect the fact that RSEs are both competent researchers (the research skills, Section 3.2) and software engineers (the software/technical skills, Section 3.1). The third pillar (communication skills, Section 3.3) forms the bridge between the former two categories, with a particular focus on the software and research cycle and the scientific process. These competencies are relevant in a broad setting and form the foundation for specific specialisations. These competencies have been chosen in order to make RSEs contribute to an open and inclusive research environment, with tools that respect their professional values.

These skills and competencies come into play in various forms: The RSEs themselves need to acquire and develop them as their career progresses (**Career level**). However, some knowledge of software and data processing is required at all academic levels and for all positions (**Academic Progression**). The relative importance of the skills and competencies also depends on the size of the RSE team (**Project team size**). Finally, different sets of skills are emphasised in the different RSE specialisations (**RSE specialisations**).

During the Paderborn workshop (deRSE23) we asked learners and novice RSEs what they would like to have learnt. The top five items mentioned were: testing, contributing to large projects, when or why to keep repositories private, high-quality software development, and finding a community. Those topics comprise combinations of the skills and competencies defined below.

3.1 Software/Technical skills

Besides skilled researchers, RSEs are also competent software engineers. As such, they ideally can solve complex software engineering problems and design software as a user-oriented, future-proof product. The technical skills required by an RSE overlap to a large extent with the common fundamental software engineering skills (see, e.g., Landwehr et al. [19]), but put greater emphasis on aspects related to achieving good scientific practice and to serving special needs of research software. In addition, a lot of RSEs are either self- or peer taught in these

skills (see, e.g., figure 14 in Barker et al. [2]). These skills include requirements analysis, design, construction, testing, program analysis, and maintenance of software. On the other hand, RSEs also know how to make research software adhere to the FAIR principles [1], and how to achieve different levels of research software reusability (see, e.g., Chue Hong [6]), while they have deeper understanding of the scientific context around the research software projects they work on. To reflect this, the technical skills listed below complement competencies regarding the standard life cycle of software development (as summarised in subsubsection 3.1.1) with RSE-specific focus skills.

3.1.1 Classical software engineering skills

To summarise the vast range of the skills a software engineer is typically equipped with, we refer to the Guide to the Software Engineering Body of Knowledge (Bourque, Fairley, and IEEE Computer Society [4]). Because research software engineering is an interface discipline, RSEs are often stronger in topics more commonly encountered in research software contexts (e.g., mathematical and engineering foundations) than in other areas (e.g., software engineering economics). However, they bring a solid level of competence in all software engineering topics. Therefore, RSEs can set and analyse software requirements in the context of open-ended, question-driven research. They can design software so that it can sustainably grow, often in an environment of rapid turnover of contributors. They are competent in implementing solutions themselves in a wide range of technologies fit for different scientific applications. They can formulate and implement various types of tests, they can independently maintain software and automate operations of the integration and release process. They can provide working, scalable, and future-proof solutions in a professional context and with common project and software management techniques, adapted to the needs of the research environment. Finally, as people who have often gained significant research experience in a particular discipline, they combine the necessary foundations from their domain with software engineering skills to develop complex software.

3.1.2 Adapting to the software life cycle (SWLC)

The traditional software development life cycle defines the stages that form the process of building a piece of software. Initial development generally involves an analytic process where requirements and ideas are gathered and analysed (requirements engineering), followed by formulating a plan to fulfil them (design) that is finally turned into running code (implementation). This is accompanied by different measures of quality control (e.g., reviews, testing), validating and verifying that things work as expected and that they continue to do so when development progresses further. Depending on the software project, this can mean a simple "Think-before-you-do", or more elaborate and formal processes. Often the development cycles are executed iteratively and incrementally. The life cycle further includes periods of deployment, maintenance and further development (software evolution), as well as software retirement. To assess the current state and needs of the software, the RSE should be familiar with different maturity metrics, e.g. the DLR application classes [24], the research software maturity model [7] or technology readiness levels (TRLs). Additionally, the research software life cycle extends the traditional life cycle with software publication. The RSE should be aware of this life cycle and be able to predict and cater to the changing needs of a software project as it moves through the stages.

3.1.3 Creating documented code building blocks (DOCBB)

The RSE should be able to create building blocks from source code that are reusable. This ranges from simple libraries of functions up to complex architectures consisting of multiple software packages. An important part of enabling code reusability is the provision of sufficient information in the form of comments within code, documentation or other means. This is vital to ensure that developers and maintainers understand what a piece of software aims to do and how to enable others to use the provided functionality. This is primarily achieved through a "clean" implementation and enhanced by documentation. Documentation ranges from commenting code blocks to using documentation (building) tools. It should be written with consideration for the different audiences who may need it depending on their goals and expertise, for example by following the Diátaxis framework [23].

3.1.4 Building distributable software (DIST)

The RSE should be able to distribute their code on their domain/language specific distribution platforms. This almost always encompasses handling/documenting dependencies with other packages/libraries. It sometimes requires knowledge of using build or package management systems to enable interoperability with other projects. In terms of usability and needs of the user community the RSE should be able to decide whether a library or a framework is the right type of program to build and distribute.

3.1.5 Use software repositories (SWREPOS)

The RSE should be able to identify and use fitting public platforms (so-called software repositories or "repos") to share the artefacts they have created and invite the public to scrutinise them in public reviews. These software repositories usually provide facilities for software development, which differentiate them from the domain repositories described later.

3.1.6 Software behaviour awareness and analysis (MOD)

We define this as a certain quality of analytical thinking that enables an RSE to form a mental model of a piece of software in a specific environment (program comprehension). Using that, an RSE should be able to make predictions about a software's behaviour. This is a required skill for common tasks such as debugging, profiling, optimising, designing good tests, or predicting user interaction. Many tools exist to help with understanding and evaluating existing code, especially from a structural point of view. An RSE should understand their output and its implications. An important facet of this capability relates to information security. RSEs need to consider the safety and integrity of personal data and other sensitive information and make sure that they do not negatively impact the integrity of their institution's network and computing infrastructure.

3.2 Research skills

3.2.1 Conducting and leading research (NEW)

RSEs are curious and able to conduct research, both on research software engineering, and on their research-wise "home domain". Senior RSEs are also able to lead research, and many RSEs have a doctorate [16]. Since RSEs often operate in different research fields, they also gain their reputation from their effectiveness in interacting with researchers from the same or other domains. Therefore, some curiosity together with a broad overview of the research field is required, as this enables the RSE to learn new methods and algorithms directly from domain peers. Similarly, a broad overview of the field of SE research and the growing field of RSE research enables the RSE to learn, apply, and teach new methods and tools for improving the way they develop software. This curiosity, together with the ability to convert it into new ideas, is also reflected when an RSE is actively trying out new tools or discovering related literature from adjacent domains. Lifelong learning is then no longer just a phrase but becomes a motivation to work.

3.2.2 Understanding the research cycle (RC)

One of the key skills that RSEs have is their understanding of how research works. They embrace being part of a larger community which, despite friendly competition, shares the common goal of gaining knowledge to disseminate it. Thereby they know that they are part of a bigger undertaking that involves many other parties in and outside their domain, and also that their software can be utilised at different stages of the research cycle by different people. They may be asked to contribute to the ethical and regulatory evaluation of a project to ensure integrity of the research performed therein. Like other researchers, RSEs are open to discussions and arguments beyond their own expertise and appreciate the underlying principles of good research, including publications, reviews and reproducibility.

3.2.3 Software re-use (SRU)

The re-use of existing assets such as libraries and pieces of code to improve efficiency and quality belongs to the fundamentals of software construction [4]. To discover software, RSEs rely on domain-specific knowledge and domain repositories, as well as research skills, discovering related software via software citations and metadata. To evaluate whether the artefacts to be re-used suit their needs, RSEs often need to consider the scientific context of their origin. For example, a paper that references the code under consideration might be crucial to validate its fitness for purpose or lack of suitability. Code that incorporates research-domain specific knowledge needs to be understood at a very detailed level and its re-use documented to meet standards of good research practice. Not only the technical compatibility needs to be understood and documented (programming languages, system interoperability), but also the underlying models and computational methods need to fit the purpose; this question often requires wider research skills and deeper understanding of the research domain at hand.

3.2.4 Software publication and citation (SP)

Another part of FAIR software is concerned with publishing new and derived works and making them available for re-use by the research community and the general public. RSEs need to have a basic understanding of common

software licence types, including proprietary and open source licences and how "copyleft" and "permissive" open source licences differ. They should also understand compatibility between different licences, and the ramifications for re-using and composing programs. Beyond that, RSEs will need to properly execute the technicalities of software publishing. These include the application of licences and copyright statements, understanding and assigning software authorship, crediting contributors, maintaining FAIR software metadata and publishing software artefacts. Finally, RSEs will need to understand the principles of software citation [26]. This concerns both the potential for reuse of their own work, which demands the provision of complete and correct up-to-date citation metadata for their software, as well as their own citation obligations deriving from building on previous work in the form of dependencies.

3.2.5 Using domain repositories/directories (DOMREP)

Almost all research software is developed within a specific scientific domain. Some software may be able to cross boundaries, but the majority will have a home domain, with which it needs to be able to interact. The RSE then needs to be aware of any domain specific repositories that will contain data sets, catalogues, and other domain specific artefacts, in addition to software. The RSE also needs to be aware of how their software can interact with the existing domain-specific data repositories. Finally, they need to be able to assess and use software repositories - domain-specific or generic - for publishing software with the relevant metadata.

3.3 Communication skills

RSEs do not work in isolation. They are embedded in a research group or work within a team of RSEs supporting particular research projects. RSEs often need to interact with and facilitate communication among colleagues, clients and contractors with a very broad spectrum of background-knowledge, specialisation, expectations, and experience whilst keeping diversity issues in mind. Communication skills are therefore crucially important. Team skills are also mentioned in common guides for SE such as the software engineering body of knowledge [4]. However, the interpersonal and organisational skills and the capacity for adaption required to work in a research setting warrants a much stronger emphasis on this field of competence.

3.3.1 Working in a team (TEAM)

Being able to work, and effectively communicate in teams is essential for RSEs. For example, RSEs need to be able to explain particular implementation choices made and may even need to defend them. Within a team of RSEs, code reviews improve knowledge transfer and increase team cohesion. The team might change on a project-to-project basis and might be comprised of colleagues with very different backgrounds including, for example, information technology (IT) staff, domain scientists and technicians working alongside software engineers. The shared values come into play and each RSE needs to ensure that these values are lived by and passed on to others. Senior RSEs may lead a team of RSEs.

3.3.2 Teaching (TEACH)

RSEs have many opportunities to teach. These range from inducting new colleagues to teaching digital skills either through short courses, for example from The Carpentries [27], or entire lecture series. RSEs may also act as mentors and consultants. Code review also includes aspects of the teaching skill.

3.3.3 Project management (PM)

The RSE should have knowledge of project management processes. At some institutes, project management tools and approaches differ between individual research groups, but it is useful if an RSE understands general structures of a PM scheme, or can bring in new ideas for improvement. Project management in research software engineering poses specific challenges (see USERS) that might require the capacity to flexibly adapt to changing conditions and deviate from common project management methods. Additionally, the RSE should know that SE offers various methods and approaches specifically tailored to management of software projects and products.

3.3.4 Interaction with users and other stakeholders (USERS)

Since research software is often developed as part of the research process itself, its requirements and specifications might change with the progression of research. Stakeholders of research software often change across different research projects or even within the course of one project. Roles in connection with research software are often in flux and diffuse. For example, a single person might be user, developer and project manager at the same time.

Often this means it is necessary for an RSE to think "outside their comfort zone", but at the same time to be able to convey their knowledge and experience to experts of other fields or persons at different hierarchy levels in a way they can understand more easily. These conditions pose specific challenges for requirements analysis, project management, training and support.

4 RSE specialisations

What we have defined above are intended to be base skills that an RSE irrespective of domain, position, and experience should know about. There is a large variety of RSEs. They specialise in different areas, some of which we want to present below. Many of the specialisations may overlap, so the same RSE might for example work on data management and open science. We categorise them into those that can be viewed as a specialisation within RSE-specific topics, while other RSEs might expand their skill set and profession to areas that are not typical for an RSE.

4.1 Specialisations within the core RSE competencies

Open science RSE Open science and FAIRness of data and software are increasingly important topics in research, as exemplified by the demand of an increasing amount of research funding agencies requiring openness. Hence, an open science RSE is required to have a deeper knowledge of (RC) and how to distribute software publicly (SRU, SP). Open Science RSEs can help researchers navigate the technical questions that come up when practising Open Science, such as "How do I make my code presentable?", "How do I make my code citable?", "What do I need to do to make my software FAIR?", or "How do I sustainably work with an (international) team on a large code base?". Like the Data-focused RSE, they have a deep understanding of research data management (RDM) topics.

Project/community manager RSEs When research software projects become larger, they need someone who manages processes and people. In practice, this concerns change management for code and documentation and community work to safeguard usability and adaptability, but also handling project governance and scalable decision-making processes. This gap can be filled by people who invest in the (PM), (USERS), and (TEAM) skills. Building a community around a research project is an important building block for sustainable software [25], so these RSEs play an important role, even if they do not necessarily touch much of the code themselves.

Teaching RSEs RSEs interested in developing their (TEACH) skill can focus on teaching the next generation of researchers and/or RSEs and will play a vital role in improving the quality of research software. They need to have a good understanding of all RSE competencies relevant to their domain and additionally should have teaching experience and training in didactics and pedagogy.

User interface/user experience designers for research software Scientific software is a complex product that often needs to be refined in order to be usable even by other scientists. To facilitate this, there are people required that specialise in the (DOCBB) and probably the (DIST) competency with a focus on making end-user facing software really reusable and hence FAIR. This task is supported by strong (MOD) skills to reason about the behaviour of potential users of the software.

4.2 Specialisations outside the core RSE competencies

${DOMAIN}-RSE While software is the common focus of all RSEs, there will be RSEs that have additionally specialised in the intricacies of one particular research domain, such as medical RSEs, digital humanities RSEs, or physics RSEs.

Data-focused RSE Data-focused RSEs work at the flourishing intersection between data science and RSE. They are additionally skilled in cleaning data and/or running data analyses and can help researchers in setting up their analysis pipeline and/or RDM solutions. When the field requires research on sensitive data or information, e.g., patient information in medicine, this RSE should have knowledge about secure transfer methods and/or ways to anonymise the data. As part of RDM, this RSE profile is able to support all stages of the research data life cycle [21], with synchronous data management processes. Those processes implement established best practices for planning and documenting of data acquisition in a data management plan (DMP), as well as for management, storage, and preservation of data, and publication and sharing of data in repositories according to the FAIR principles [32].

Research infrastructure RSE This RSE has a special interest in SysOps and system administration and sets up IT infrastructures for and with researchers. Therefore, this specialisation on the one hand requires a deep knowledge of physical computer and network hardware and on the other hand knowledge about setup and configuration of particular server software, e.g., setup of virtual machines on hypervisors or the planning and setup of compute server clusters for special purposes, e.g., ML. As an interface between the researchers and the infrastructure, they take care of user management, access permissions, and configuration of required services.

HPC-RSE RSEs with a focus on High-Performance Computing (HPC) have specialist knowledge about programming models that can be used to efficiently undertake large-scale computations on parallel computing clusters. They may have knowledge of (automatic) code optimisation tools and methods and will understand how to write code that is optimised for different types of computing platforms, leveraging various efficiency related features of the target hardware. They are familiar with HPC-specific package managers and can build dependencies from sources. They also understand the process of interacting with job scheduling systems that are often used on HPC clusters to manage the queuing and running of computational tasks. HPC-focused RSEs may be involved with managing HPC infrastructure at the hardware or software level (or both) and understand how to calculate the environmental impact of large-scale computations. Their knowledge of how to run HPC jobs and write successful HPC access proposals can be vitally important to researchers wanting to make use of HPC infrastructure.

ML-RSE The development of research software based on ML requires additional specialised theoretical background and experienced handling of appropriate software in order to produce meaningful results. This involves knowledge about data analysis and feature engineering, metrics that are involved in ML, ML algorithm selection and cross validation, and knowledge in mathematical optimisation methods and statistics. Here, we use ML in a broad sense of machine-based learning including deep learning, reinforcement learning, neuro-symbolic learning and similar.

ML-RSEs analyse and check the suitability of an algorithm. They check if it fulfils the needs of a certain task and they play a central role in deciding on and selecting ML libraries for a given task. The increasing usage of ML in numerous scientific areas with social impact involves an emphasised awareness and consideration of possible influences and biases. At the intersection of data science [11] and data-focused RSEs, the complex way of solving problems utilising ML calls for this separate specialisation.

Legacy RSEs Research software may have evolved over generations of researchers without change management or governance processes, while software "ecosystems" (e.g., programming languages, frameworks, operating systems) constantly evolve. This may lead to the emergence of legacy code that is still actively used. To safeguard continued usability and adoption, these RSEs have experience in working with code written in language standards and on software stacks considered deprecated by their communities. Adaption of existing, large-scale codebases to evolving dependencies (DIST) or changing hardware (HPC; see the HPC-RSE specialisation) may require mastery in refactoring techniques and in the usage of specialised code transformation tools.

Web-development RSE This RSE is skilled in the development of web applications and/or mobile apps. They have expertise in one or more of frontend development, backend development and the design or implementation of APIs, for example to support research data portals or big research projects. Since a lot of web services for research may be accessible to a large audience or even to the public, this RSE is also familiar with aspects relating to cybersecurity, usability and accessibility. Not only do they need to balance these concerns while adhering to their values from Section 2, but they also need to efficiently communicate the decisions made to stakeholders.

Legal-RSE RSEs are often the go-to person for questions about software licensing, in particular when mixing software components that use different licences. But with the rising requirements from legislation, we foresee the need for RSEs that still have a background in RSE but extend it with a knowledge of legal processes that cover corner cases and go beyond applying Best Practice guides. These requirements may arise in the area of publication of research software, as this also requires knowledge about particular laws or regulatory frameworks concerning data protection, like the General Data Protection Regulation (GDPR) within the European Union (EU) [28]. Another area are legal aspects of cybersecurity and export control in science and research (see [8] for Germany). Legal-RSEs focus on facilitating the achievement of technically feasible solutions, while adhering to regulatory mandates. They are able to communicate and collaborate effectively with lawyers.

5 Future work

This list and description of competencies is a first step to finding common ground around which to structure curricula, institutions, and teachers in this framework. An omission that we found and that we would like to highlight in order to spark a community discussion is that RSEs that choose explicitly a science-supporting role outside of research will not be eligible for funding under the statutes of many funding organisations that require at least a PhD.

To alleviate this and to give RSEs in leadership positions a means to become eligible for funding themselves, since completion of scientific training is often a requirement [12], we see two possible parts of a solution. One is to allow for doctorates primarily based on software contributions to the scientific community. Secondly, we propose the introduction of new, standardised certificates and to officially accept them as PhD-equivalent concerning eligibility to be a Principal Investigator (PI). Beyond this discussion, a diverse set of publications on the topic RSE teaching is already in the making.

Within this set, we will work next on how to institutionalise education. In that publication, we will detail how we organise our institutions and what qualifications our teachers need to have in order to effectively communicate our values. We will put forward ideas on how to build up bachelor's and master's programmes, of which a glimpse can already be found in FIXMEPROPERCITATION. We will show how we intend to provide the necessary continuous education for RSEs after graduation, and we will connect that with the integration of RSEs into a mesh of community networks aimed at supporting research, while providing them with an inclusive social network that further facilitates lifelong learning. That publication will again intentionally be free of regional specifics, to also serve as a blueprint that other national RSE societies can build upon.

Online resources for courses are another important building block. This is the general intention of the learn-and-teach project [30]. Surveying and curating of existing resources is not carried out as a traditional publication, but it is made available as a continuously-evolving online resource at [30].

And finally, we plan to formulate a call to action, building on the previously mentioned publication on the necessary institutions, that spells out everything that is required to best support the continuous need for young RSEs to support digital science specifically in Germany.

6 Conclusion

This paper started from a community workshop at deRSE23 in Paderborn where people working in RSE related fields got together to figure out structures and ideas for educating newcomers to this field. One outcome of this diverse gathering is that RSEs from differing fields gather around similar core concepts, At the same time they share a vision of how to renew scientific research practice making extensive use of digital tools. In this publication, we have tried to formalise these concepts. We have formulated a set of values that guide our actions in society, manifestly making RSEs part of the scientific community that shares the ideals of good scientific practice. At the same time, being close to software engineers, we cherish that we have to take responsibility for our tools. We listed core competencies that have been intentionally formulated abstractly without referencing any particular information-processing device. As expected, we have drawn equally upon notions from SE and other research fields, but found that we likewise require teamwork capabilities. We detailed these competencies in various dimensions and found that a different amount is required in different positions and scientific domains. Using this, we proposed recommendations for organisations to foster the development of these competencies.

The gathered values and competencies form a common denominator that unifies RSEs and enables them to identify with this domain, in the knowledge that it is already or will soon become critically important for many areas of science. These competencies at the intersection of research and SE, coupled with a firm belief in team processes, make RSEs sought after on the job market and their values make them responsible members of a digital society. The result is a qualification profile which is highly attractive for young people.

At an institutional level, research performing organisations have a growing interest in fostering RSE training to support the use of FAIR data and FAIR software in the academic world, a direction determined by new incentives created by scientific journals and librarians. How we update existing institutions and set up new ones that provide this education will be the topic of a follow-up paper.

Contribution details

Heidi Seibold came up with the original idea for the deRSE23 workshop in Paderborn. Heidi Seibold, Jeremy Cohen, Florian Goth, Renato Alves, Jan Philipp Thiele, and Samantha Wittke organised the deRSE23 workshop. We thank all the participants of this community workshop! Toby Hodges conceptualised and organised the un-deRSE23 workshop together with Jan Philipp Thiele and Florian Goth. We also thank all the participants of this follow-up community workshop! Jeremy Cohen, Gerasimos Chourdakis, Magnus Hagdorn, Jean-Noël Grad, Jan Philipp Thiele, and Matthias Braun organised the deRSE24 workshop in Würzburg. We are also grateful to the participants of this third community workshop! Heidi Seibold, Jeremy Cohen, Florian Goth, Renato Alves, Jan Philipp Thiele, Jan Linxweiler, Jean-Noël Grad, and Samantha Wittke contributed the initial draft. Florian Goth supervised the project and did the project administration. Jean-Noël Grad designed and implemented the software tooling for the collaborative writing of this manuscript on GitHub. Everybody contributed to the final review and editing.

The CRediT system [5] is far too generic to adequately describe the contributions of everybody in various workshops, spread over a two year period. While everybody contributed to the discussion formulating and refining the ideas, and to collaboratively writing and/or reviewing and editing the entirety the script, some parts merit special mention. Renato Alves quickly jumped in to host the first deRSE23 workshop to take over from a sick organiser. Matthias Braun contributed early versions of the specialisations and also contributed to the survey. Leyla Jael Castro contributed to the initial draft of the example career path, and provided helpful insights in discussions on metadata. Gerasimos Chourdakis' contributions to the paper are numerous (extensively editing large parts of the initial draft), but he especially wrote first drafts for clarifying the relationship of the RSE competencies to the SE competencies. He also designed the "Learning and teaching RSE" website [30]. Simon Christ helped with typesetting and contributed the competencies' symbols and the spell-checker script. Jeremy Cohen drafted the initial introduction and contributed the tables for RSEs in centralised RSE departments. Stephan Druskat contributed parts on proper software citation and publication and sharpened various RSE specialisations. Fredo Erxleben contributed to early discussions of the paper and added the contributions by the Helmholtz Association. Jean-Noël Grad contributed initial drafts for the ELIXIR framework and the section on the work of the HPC certification forum as well as numerous other contributions to the BibTeX infrastructure and the GitHub actions. Magnus Hagdorn drafted and supervised the ethics and values section for an RSE and made sure that these values are reflected in the competencies of RSEs. Toby Hodges contributed parts on the Carpentries, and helped steer the curriculum discussion. Guido Juckeland contributed experiences from his first RSE course for students. Dominic Kempf drafted the first version of the example curriculum. Anna-Lena Lamprecht helped with proper wording, especially with awareness about established SE terminology, that was misused earlier. Jan Linxweiler drafted various RSE specialisations and made sure that clean coding techniques got their due recognition. Frank Löffler rewrote numerous parts to be actually legible, and helped with preparation for the final steps of a de-RSE position paper. Michele Martone wrote the first draft of the environmental sustainability section. Moritz Schwarzmeier drafted the categorisation of the specialisations. Heidi Seibold contributed the idea and started everything. Jan Philipp Thiele drafted initial parts of the technical pillar of the RSE competencies, and represented the project on numerous discussions. Harald von Waldow contributed to initial drafts of the Masters program and contributed his knowledge to the explainability of computer simulations. Samantha Wittke contributed the parts on CodeRefinery and how to reach out to new RSEs. Florian Goth has the pleasure of being grateful to all collaborators in this project for contributing their time and knowledge into this project!

Acknowledgements

We thank all the contributors to the teachingRSE project!

We appreciate the comments and suggestions from Yves Vincent Grossmann, Wilhelm Hasselbring, and Bernhard Rumpe.

References

[1] Michelle Barker et al. "Introducing the FAIR Principles for research software". In: *Scientific Data* 9.1 (2022-10), p. 622. ISSN: 2052-4463. DOI: 10.1038/s41597-022-01710-x.

[2] Michelle Barker et al. *Software and skills for research computing in the UK*. Tech. rep. Software Sustainability Institute, 2023-11. DOI: 10.5281/zenodo.10473185.

[3] Alisa Bokulich and Wendy Parker. "Data models, representation and adequacy-for-purpose". In: *European Journal for Philosophy of Science* 11.1 (2021-01-29), p. 31. ISSN: 1879-4920. DOI: 10.1007/s13194-020-00345-2.

[4] Pierre Bourque, Richard E. Fairley, and IEEE Computer Society. *Guide to the Software Engineering Body of Knowledge (SWEBOK(R)): Version 3.0.* 3rd ed. Washington, DC, USA: IEEE Computer Society Press, 2014-01. 346 pp. ISBN: 978-0-7695-5166-1.

[5] Amy Brand et al. "Beyond authorship: attribution, contribution, collaboration, and credit". In: *Learned Publishing* 28.2 (2015), pp. 151–155. DOI: https://doi.org/10.1087/20150211. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1087/20150211. URL: https://onlinelibrary.wiley.com/doi/abs/10.1087/20150211.

[6] Neil Chue Hong. "Minimal information for reusable scientific software". In: *2nd Workshop on Sustainable Software for Science: Practice and Experiences (WSSSPE2)* (New Orleans, Louisiana, USA, 2014-11-16). figshare, 2014-07. DOI: 10.6084/m9.figshare.1112528.

[7] Deekshitha et al. *RSMM: A Framework to Assess Maturity of Research Software Project.* arXiv. 2024-06. DOI: 10.48550/arXiv.2406.01788. arXiv: 2406.01788 [cs.SE].

[8] *Export Control in Science & Research.* Federal Office for Economic Affairs and Export Control (BAFA). URL: https://www.bafa.de/SharedDocs/Downloads/EN/Foreign_Trade/ec_awareness_academia.html (visited on 2023-08-04).

[9] Deutsche Forschungsgemeinschaft. *Guidelines for Safeguarding Good Research Practice.* Code of Conduct version 1.1. 2022-04. DOI: 10.5281/zenodo.6472827.

[10] Roman Frigg and Julian Reiss. "The philosophy of simulation: hot new issues or same old stew?" In: *Synthese* 169.3 (2009-08-01), pp. 593–613. ISSN: 1573-0964. DOI: 10.1007/s11229-008-9438-z.

[11] R. Stuart Geiger et al. *So you want to start a data science institute? Achieving sustainability.* Software Sustainability Institute. URL: https://www.software.ac.uk/blog/2018-04-05-so-you-want-start-data-science-institute-achieving-sustainability (visited on 2023-08-04).

[12] German Research Foundation. *Merkblatt Programm Sachbeihilfe.* 2023-06. URL: https://www.dfg.de/formulare/50_01/50_01_en.pdf (visited on 2023-11-14).

[13] Don Gotterbarn, Keith Miller, and Simon Rogerson. "Software Engineering Code of Ethics is Approved". In: *Communications of the ACM* 42.10 (1999-10), pp. 102–107. ISSN: 0001-0782. DOI: 10.1145/317665.317682.

[14] Morane Gruenpeter et al. *Defining Research Software: a controversial discussion.* Summary Report of FAIR4RS Subgroup 3 activity and discussion Version 1. FORCE11, Research Data Alliance, Research Software Alliance, 2021-09. DOI: 10.5281/zenodo.5504016.

[15] Simon Hettrick. *A not-so-brief history of Research Software Engineers.* Software Susitainability Institute. 2016-08. URL: https://www.software.ac.uk/blog/2016-08-17-not-so-brief-history-research-software-engineers-0 (visited on 2023-06-16).

[16] Simon Hettrick et al. *International RSE Survey 2022.* Version v0.9.3. 2022-08. DOI: 10.5281/zenodo.7015772. URL: https://doi.org/10.5281/zenodo.7015772.

[17] Paul Humphreys. *Extending Ourselves: Computational Science, Empiricism, and Scientific Method.* Oxford University Press, 2004-08-12. ISBN: 978-0-19-978596-4. DOI: 10.1093/0195158709.001.0001.

[18] Anna-Lena Lamprecht. *Thema im Fokus: Forschungssoftware und Research Software Engineering (RSE).* GI-Radar. 2024-02. URL: https://gi-radar.de/351-research-software/.

[19] Carl Landwehr et al. "Software Systems Engineering programmes a capability approach". In: *Journal of Systems and Software* 125 (2017), pp. 354–364. ISSN: 0164-1212. DOI: 10.1016/j.jss.2016.12.016.

[20] Netherlands eScience Center. *What Is a Research Software Engineer? A Definition by the Netherlands eScience Center.* Tech. rep. Version 1.0. 2023-06. DOI: 10.5281/zenodo.7994286.

[21] Abraham Nieva de la Hidalga et al. "The ENVRI Reference Model". In: *Towards Interoperable Research Infrastructures for Environmental and Earth Sciences.* 2020-07, pp. 61–81. ISBN: 978-3-030-52828-7. DOI: 10.1007/978-3-030-52829-4_4.

[22] Wendy S. Parker. "Evidence and Knowledge from Computer Simulation". In: *Erkenntnis* 87.4 (2022-08), pp. 1521–1538. ISSN: 1572-8420. DOI: 10.1007/s10670-020-00260-1.

[23] Daniele Procida. *Diátaxis documentation framework.* URL: https://diataxis.fr/.

[24] Tobias Schlauch, Michael Meinel, and Carina Haupt. *DLR Software Engineering Guidelines.* Tech. rep. Version 1.0.0. German Aerospace Center, 2018-08. DOI: 10.5281/zenodo.1344612. Translation of *Software-Engineering-Empfehlungen des DLR.* Tech. rep. Version 1.0.0. German Aerospace Center, 2018-08. DOI: 10.5281/zenodo.1344608.

[25] Judith Segal. "Some challenges facing software engineers developing software for scientists". In: *Proceedings of the 2009 ICSE Workshop on Software Engineering for Computational Science and Engineering* (Vancouver, British Columbia, Canada, 2009-05-23/2009-05-23). IEEE, 2009-05. ISBN: 978-1-4244-3737-5. DOI: 10.1109/secse.2009.5069156.

[26] Arfon M. Smith et al. "Software Citation Principles". In: *PeerJ Computer Science* 2.e86 (2016). ISSN: 2376-5992. DOI: 10.7717/peerj-cs.86.
[27] The Carpentries. *The Carpentries*. URL: https://carpentries.org (visited on 2023-06-16).
[28] The European Parliament and the Council of the European Union. "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)". In: *Official Journal of the European Union* 59.L 119 (2016-05). ISSN: 1977-0677. URL: https://eur-lex.europa.eu/eli/reg/2016/679/2016-05-04.
[29] *The Missing Semester of Your CS Education*. Massachusetts Institute of Technology. URL: https://missing.csail.mit.edu/ (visited on 2023-07-21).
[30] The teachingRSE project. *Learning and teaching RSE*. URL: https://de-rse.org/learn-and-teach/ (visited on 2024-02-22).
[31] Igor Wiese, Ivanilton Polato, and Gustavo Pinto. "Naming the Pain in Developing Scientific Software". In: *IEEE Software* 37.4 (2020-07), pp. 75–82. ISSN: 1937-4194. DOI: 10.1109/MS.2019.2899838.
[32] Mark D. Wilkinson et al. "The FAIR Guiding Principles for scientific data management and stewardship". In: *Scientific Data* 3.1 (2016-03), p. 160018. ISSN: 2052-4463. DOI: 10.1038/sdata.2016.18.
[33] Eric Winsberg. "Sanctioning Models: The Epistemology of Simulation". In: *Science in Context* 12.2 (1999-07), pp. 275–292. ISSN: 1474-0664, 0269-8897. DOI: 10.1017/S0269889700003422.

Glossary

software publication the practice of long-term archiving software artifacts with software metadata under a permanent identifier. 6

SysOp system administrator in charge of a computing infrastructure. 9

Skill codes

DIST Building distributable software. 9, 10

DOCBB Creating documented code building blocks. 9

MOD Software behaviour awareness and analysis. 9

PM Project management. 8

RC Understanding the research cycle. 8

SP Software publication and citation. 8

SRU Software re-use. 8

TEACH Teaching. 9

TEAM Working in a team. 8

USERS Interaction with users and other stakeholders. 8

Acronyms

DMP data management plan. 9

EU European Union. 10

FAIR Findability, Accessibility, Interoperability and Reusability. 2, 4, 5, 7–9, 11

GDPR General Data Protection Regulation. 10

HPC High-Performance Computing. 9, 10

IT information technology. 7, 9

MIT Massachusetts Institute of Technology. 2

ML machine learning. 2, 9

PI Principal Investigator. 10

RDM research data management. 8, 9

SE software engineering. 2–4, 7, 11

STEM science, technology, engineering and mathematics. 2

UK United Kingdom. 2